

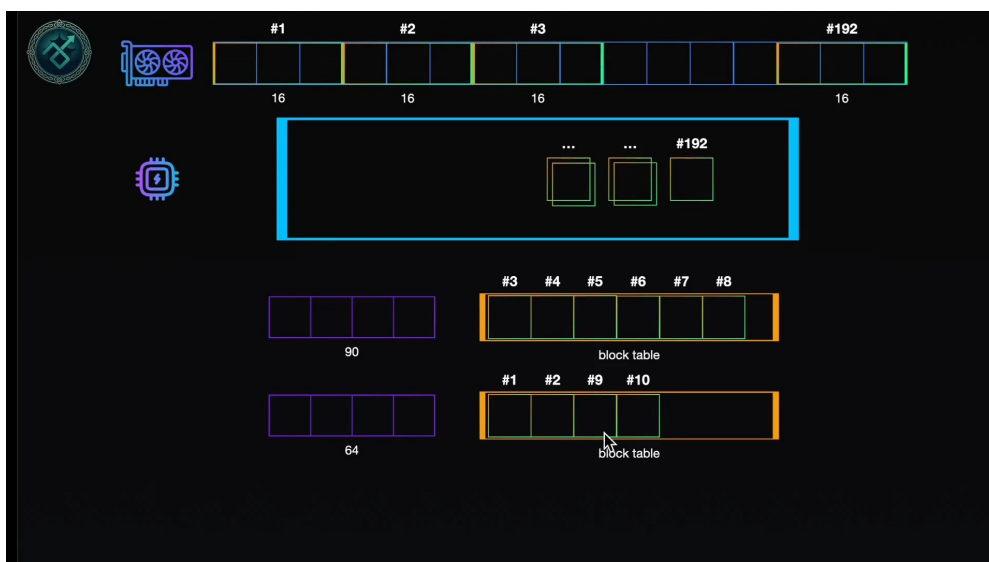
课程笔记

PagedAttention: 为什么它能提升 GPU 显存利用率

把操作系统的”内存分页”搬进 KV Cache 管理

lecture-to-notes 自动讲义

2026 年 6 月 20 日



视频作者/频道: 向量隐修会

发布平台: Bilibili

视频时长: 约 13 分钟

视频链接: <https://www.bilibili.com/video/BV1vKzuBmEWk>

目录

1 引言：重点在 Paged，不在 Attention	2
1.1 本章小结	2
2 回顾：KV Cache 怎么占显存	2
3 朴素存储的两个问题	2
3.1 问题一：空间浪费	2
3.2 问题二：显存碎片化	2
3.3 本章小结	3
4 PagedAttention：分块、池子与 block table	3
5 它如何同时治好两个问题	5
6 总结与延伸	6
6.1 作者的总结	6
6.2 笔者的延伸：四讲连起来看	6
6.3 拓展阅读	6

1 引言：重点在 Paged，不在 Attention

PagedAttention 是一个容易让人误解的名字——重点不在 *Attention*，而在 *Paged*。

一句话定义

PagedAttention 不是一种注意力机制，而是一种在显存上更高效地存储 KV Cache 的手段。它把物理显存切成一个个固定大小的块，给块编号、放进一个池子，用多少拿多少、用完放回。由此解决了“显存需要连续分配”以及“显存碎片化”两个问题。直观效果：同一张显卡有了更高的吞吐量，同一时段能服务更多的人。

要理解它，得先回到上一讲的 KV Cache，看清楚它是怎么占用显存的。

1.1 本章小结

PagedAttention 解决的是 KV Cache 的显存管理问题。下面先看朴素存储的两个毛病，再看分块方案。

2 回顾：KV Cache 怎么占显存

以“床前明”为例：第一次前向算出“床前明”三个字的 Key/Value 向量（合并存储，记为每个字的一个 KV Cache 框），并预测出下一个字“月”。存储时，先向 GPU 请求一段显存来放这些 KV Cache；之后每输出一个新 token（月 → 光 → …），就往这段显存里追加它的 KV Cache，直到停止输出。

为什么显存要“一次请求一大段”

分配显存是一个非常耗时的操作。如果每生成一个 token 就重新分配一次，会大大拉低推理速度，不可接受。所以实际做法是一次性按某个固定大小请求好，再往里填。问题正是从这个“固定大小”来的。

3 朴素存储的两个问题

3.1 问题一：空间浪费

模型的输出长度是不确定的：问“ $1+1=?$ ”输出“2”很短；让它“翻译一篇长文”则很长。由于分配显存耗时、只能按固定大小分配，而且必须迁就最长的可能输出（如 2048）。于是对那些实际很短的请求，分配的大片显存大量被浪费——6 个字符只用 6 格，其余全空着却被占住。

3.2 问题二：显存碎片化

那把固定块改小（如 1024，填满再分新的）行不行？还是不行：

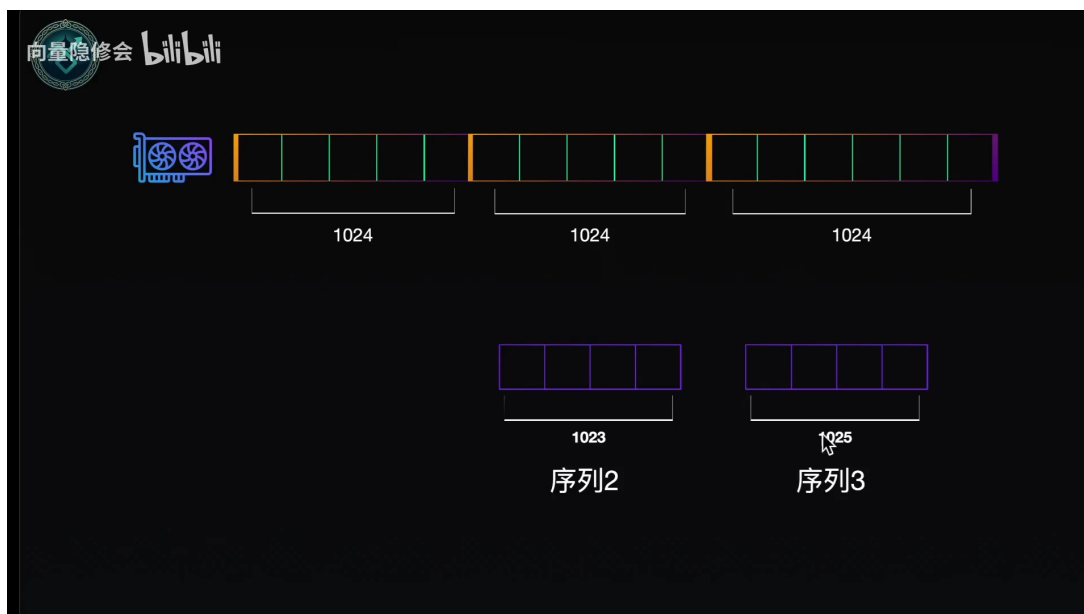


图 1: 碎片化: 显存分成 3 段、每段 1024。序列 2 (长 1023) 占了中间段; 当序列 1 结束、其占的段被释放后, 来了序列 3 (长 1025) ——虽然两段空闲共 2048, 但它们不连续, 放不下需要连续空间的 1025。有足够的总空间, 却存不下。²

碎片化的本质

朴素方案要求 KV Cache 存在**底层连续**的物理显存上。随着序列不断申请、释放, 完整的显存被切成零散的空闲碎片, 难以被高效利用——这就是显存碎片化。

3.3 本章小结

朴素存储有两病: 按最长分配导致**浪费**, 连续分配导致**碎片化**。PagedAttention 用”分块 + 间接映射”一并治好。

4 PagedAttention: 分块、池子与 block table

做法借鉴操作系统的内存分页:

²视频画面时间区间: 00:05:06–00:06:22。

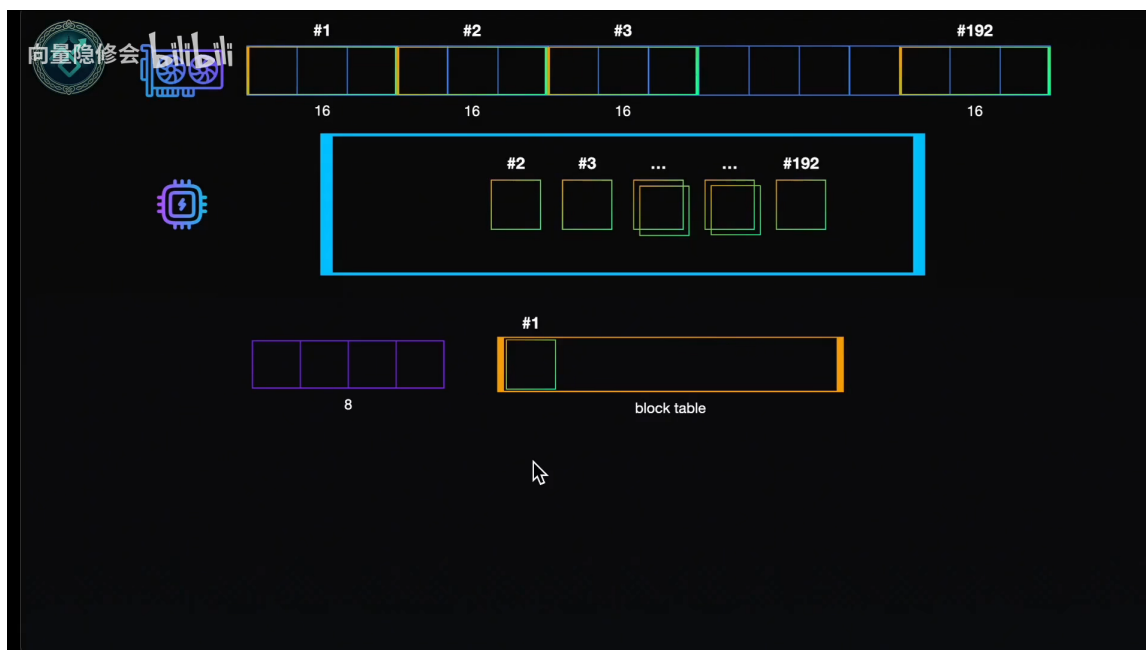


图 2: PagedAttention 的三件套：把整段显存按固定大小（如每块 16）切成 #1~#192 个块；程序里有一个池子存放所有空闲块的编号；每段输入分到一个 **block table**（数组），按需从池子取块编号填进去。图中一段长 8 的输入只需 1 个块（块大小 16，够用）。⁴

运行机制

- **块 (block)**: 显存被切成的固定大小单元；编号只是逻辑标识，不是实际空间。
- **池子 (pool)**: 存放当前空闲可用块编号的集合。
- **block table**: 每个序列一张表，记录它用了哪些块的编号。
- **动态增长**: 输入 8 个 token 用块 #1；继续输出到 17 超过 16，就从池子再取一个块 #2；序列长 90 则需 $\lceil 90/16 \rceil = 6$ 个块。
- **释放**: 序列服务结束，把它的块编号放回池子，底层物理空间即可被复用。

关键: 给序列分配“块”只是往 block table 里填编号，并不触发底层物理内存的分配——这是一个非常轻量级的操作，对推理性能影响极小。

⁴视频画面时间区间：00:06:51–00:08:22。

5 它如何同时治好两个问题

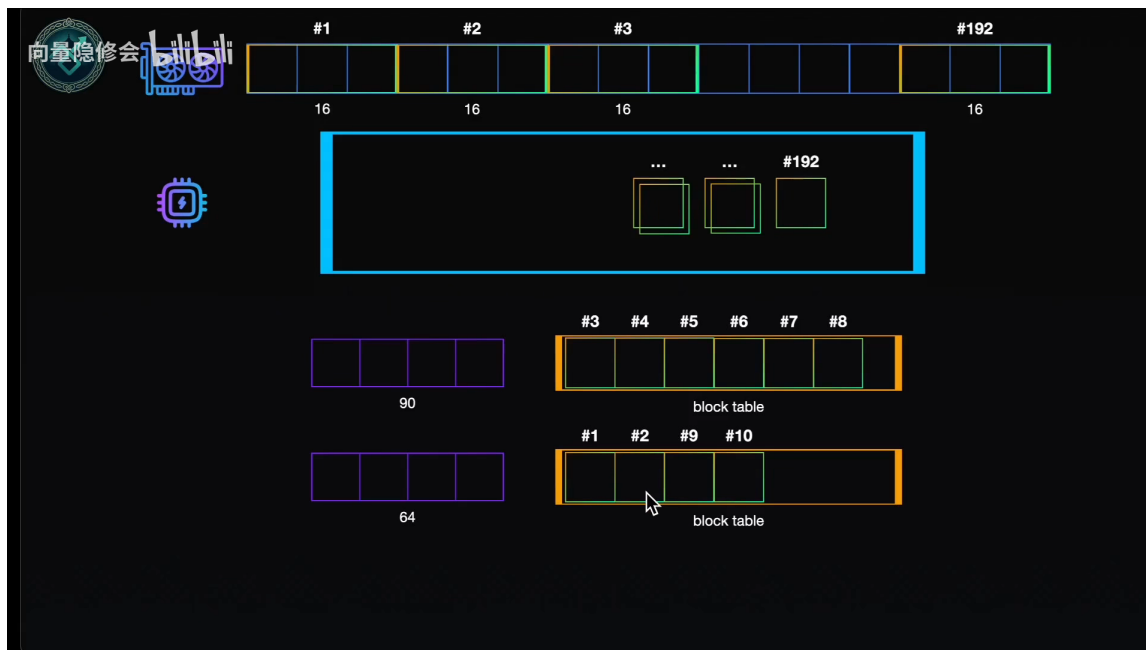


图 3: 多序列复用: 序列 (长 90) 拿到块 #3~#8; 序列 (长 64) 只需 4 个块, 于是把刚释放回池子的 #1、#2 与新的 #9、#10 拼起来用。在 block table 看来 #1#2#9#10 是连续的, 底层物理是否相邻无所谓、也感知不到。⁶

- **治浪费**: 分配以”块”为粒度。一段只有 4 个 token 的输出, 也只分 1 个块 (16 单位), 最多浪费 12 个——远小于按 2048 分配的损失。
- **治碎片化**: 碎片化的根源是”要求底层连续”。现在程序只面对 block table 里逻辑连续的编号, 底层这些块的物理位置可以四处分散; 只要池子里还有空闲块, 就能拼出任意长度, **不再需要一整段连续空间**。

下面把这层”逻辑连续、物理分散”的间接映射画出来:

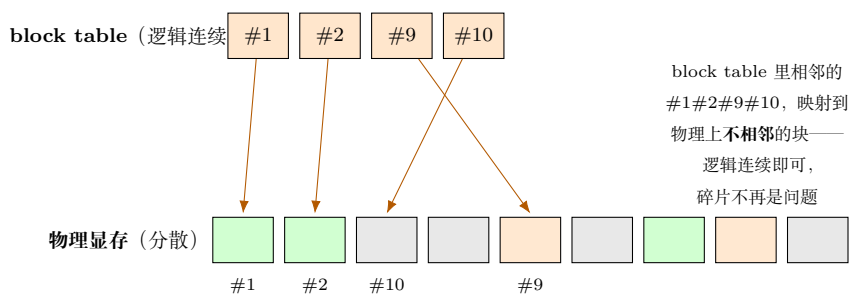


图 4: 逻辑块到物理块的间接映射 (笔记自绘): block table 提供”逻辑连续”的视图, 底层物理块可以任意分散。这与操作系统用页表把连续虚拟地址映射到分散物理页是同一思想。

⁶视频画面时间区间: 00:10:07-00:11:32。

6 总结与延伸

6.1 作者的总结

模型输出不确定、有长有短；因分配显存耗时、不能动态分配，只能按**最长**来分配大小——由此带来**空间浪费与碎片化**两个问题。PagedAttention 用**分块**的方式解决：之所以叫“Paged”，正是因为这里的“块”对应 **block**，它借鉴了操作系统里的**内存分页（paging）**——内存分页里那个单元就叫 page。

6.2 笔者的延伸：四讲连起来看

- **系列闭环**：自注意力怎么算（第一讲）→ KV Cache 存历史、免重算（第二讲）→ FlashAttention 省单次注意力的显存 IO（第三讲）→ PagedAttention 管好 KV Cache 占的那块显存（本讲）。一条从“计算”到“存储管理”的完整推理优化链。
- **操作系统的智慧迁移**：虚拟内存的“页表 + 分页”在这里几乎原样复用——用一层间接映射，把“连续性需求”和“物理布局”解耦。这是计算机系统里反复出现的强大思想。
- **为什么能服务更多人**：省下来的显存可以容纳更多并发序列（更大 batch），所以同一张卡的吞吐量更高。这是 vLLM 等推理框架的核心机制之一。
- **一句话记忆**：显存切块、按需取还；逻辑连续、物理分散；治浪费、灭碎片。

6.3 拓展阅读

- Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention* (vLLM), SOSP 2023。
- 关键词：PagedAttention、vLLM、KV Cache、Memory Fragmentation、OS Paging / Virtual Memory。
- 视频原址：<https://www.bilibili.com/video/BV1vKzuBmEWk>（“向量隐修会”注意力优化系列）。